

Documento de Requisitos

Sistema de Cálculo de Notas

1. Visão geral

Sistema para cadastrar alunos e suas notas, listar os dados cadastrados e calcular a média da turma. O programa original em COBOL é a base desta especificação, que o descreve e propõe uma evolução com persistência em banco de dados.

2. Requisitos funcionais

ID	Requisito
RF01	O sistema deve permitir cadastrar um aluno informando nome e nota.
RF02	O sistema deve permitir cadastrar até N alunos por turma (N = 5 na versão original).
RF03	O sistema deve validar que a nota esteja entre 0 e 10.
RF04	O sistema deve listar todos os alunos cadastrados com suas notas.
RF05	O sistema deve calcular e exibir a média da turma.
RF06	O sistema deve aceitar notas decimais (ponto ou vírgula).
RF07 (evolução)	O sistema deve permitir editar e excluir um aluno cadastrado.
RF08 (evolução)	O sistema deve persistir os dados em banco de dados.
RF09 (evolução)	O sistema deve indicar a situação do aluno (aprovado/reprovado) com base em uma nota mínima configurável.

3. Requisitos não funcionais

ID	Categoria	Requisito
RNF01	Usabilidade	Mensagens de entrada e erro devem ser claras e em português.
RNF02	Confiabilidade	Entradas inválidas (texto no lugar de número, nota fora da faixa) não devem encerrar o programa.
RNF03	Desempenho	O cálculo da média e a listagem devem responder em menos de 1 segundo para até 1.000 alunos.
RNF04	Manutenibilidade	O código deve ser modular (classes separadas por responsabilidade) e seguir a PEP 8.
RNF05	Portabilidade	O sistema deve rodar em Windows, Linux e macOS com Python 3.10 ou superior.
RNF06	Precisão	A média deve ser exibida com 2 casas decimais.
RNF07 (evolução)	Segurança	Os dados persistidos devem ter acesso restrito e backup periódico.
RNF08	Testabilidade	A lógica de cálculo deve ser coberta por testes unitários.

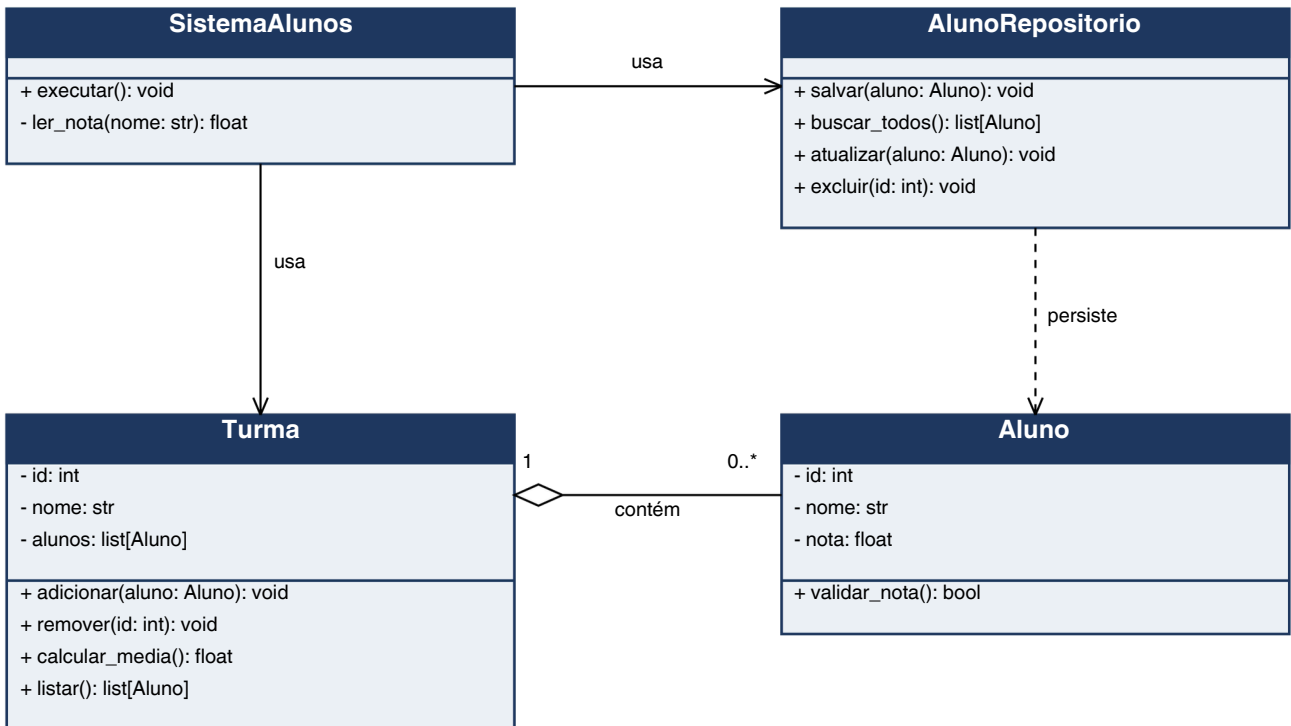
4. Regras de negócio

RN01: $\text{média} = \text{somas das notas} \div \text{quantidade de alunos}$.

RN02: nota válida está no intervalo de 0 a 10.

RN03: nome do aluno é obrigatório e não pode ser vazio.

5. Diagrama de classes



Turma contém zero ou mais Alunos (agregação). SistemaAlunos usa Turma e AlunoRepositorio; AlunoRepositorio persiste objetos Aluno.

6. Entidades e tabelas possíveis

Versão mínima (cobre o programa atual)

Tabela	Campos principais
TURMA	id (PK), nome, ano_letivo
ALUNO	id (PK), nome, turma_id (FK para TURMA)
NOTA	id (PK), aluno_id (FK para ALUNO), valor (DECIMAL(4,2)), data_lancamento

Versão ampliada (se o sistema crescer)

Tabela	Campos principais
DISCIPLINA	id, nome, carga_horaria
PROFESSOR	id, nome, email
MATRICULA	id, aluno_id, turma_id, data_matricula
AVALIACAO	id, disciplina_id, descricao, peso, data
NOTA	id, matricula_id, avaliacao_id, valor

Relacionamentos principais: Turma 1:N Aluno (versão mínima); Aluno N:N Turma por meio de Matrícula (versão ampliada); Aluno 1:N Nota. Recomenda-se uma restrição CHECK (valor BETWEEN 0 AND 10) na coluna da nota, para garantir a RN02 também no banco.

7. Bancos de dados indicados

Banco	Quando usar
SQLite	Melhor escolha para este projeto: já vem com o Python (módulo sqlite3), não exige instalação e serve bem para estudo e uso local.
PostgreSQL	Se virar um sistema web com vários usuários simultâneos; é robusto, gratuito e tem ótimo suporte a restrições e integridade.
MySQL / MariaDB	Alternativa muito comum em hospedagens e sistemas web.
SQL Server / Oracle	Faz sentido em ambientes corporativos ou legados que já usam essas plataformas (comum onde COBOL existe).

Como os dados são tabulares e têm relacionamentos claros (aluno, turma, notas), bancos relacionais são o caminho natural. Bancos NoSQL, como o MongoDB, não trazem vantagem neste cenário.